

Today 3:57 PM

</> ai(1).py
Code

</> board(1).py
Code

</> game(1).py
Code

</> main(1).py
Code

README(1).md
Document

I am working on a Connect Four Vibe Coding lab.

I have uploaded the README.md, main.py, game.py, board.py, and ai.py files from my existing project.

IMPORTANT: Do NOT modify any code yet.

First inspect the existing project and explain:

1. How the program starts from main.py.
2. How the game flow works in game.py.
3. How the board is represented and how discs are placed in board.py.
4. How winner detection currently works in board.py.
5. How the AI currently chooses its moves in ai.py.
6. Identify the existing bugs or incomplete functionality related to Tasks 1–4 in README.md.

For now, only analyze and explain the existing implementation. Do not generate replacement code and do not make any changes.

I need to understand the existing code before making changes.

I inspected all five uploaded files. I will **not modify or generate any code** at this stage.

1. How the program starts — main.py

main.py is very small. It imports the Game class from game.py and, when the file is run directly, creates a Game object and calls its run() method. </> main

So the execution flow is:

</> Plain text

```
python main.py
↓
main.py
```

```
↓
Game()
↓
Game.run()
↓
Connect Four starts
```

The README also confirms that `main.py` is the **entry point** and that the intended way to start the game is `python main.py` . [📖 README](#)

2. How the game flow works — `game.py`

The `Game` class initializes three important things:

- `self.board = Board()` → creates the game board
- `self.ai = AI()` → creates the AI
- `self.turn = "X"` → player X goes first [🔗 game](#)

Main loop

`run()` continuously performs the following:

🔗 Plain text

```
Display board
↓
Whose turn?
├─ X → ask human for column
└─ O → ask AI for column
↓
Try to place disc
↓
Check winner
↓
Check draw
↓
Switch turn
↓
Repeat
```

For the human player, it accepts a column from 1–7, converts it to a zero-based index using:

🔗 Plain text

```
entered column - 1
```

It also allows `q` to quit. Invalid non-numeric input prints "Enter a column number." and asks again. `</> game`

For the AI's turn, it simply asks:

`</> Plain text`

```
self.ai.choose_column(self.board)
```

for a column. `</> game`

Then the selected column is passed to:

`</> Plain text`

```
self.board.drop(col, self.turn)
```

If the move is invalid or the column is full, `drop()` returns `None`, and the game prints "Column unavailable." `</> game`

After a successful move:

1. `winner()` is checked.
2. If there is a winner, the game prints the final board and stops.
3. Otherwise, `full()` checks for a draw.
4. If neither happened, the turn changes between X and O. `</> game`

Important observation

The overall game loop is already reasonably structured. The major incomplete behaviour is not that the loop is missing entirely; rather, some of the functions it relies on are incomplete.

3. Board representation and disc placement — `board.py`

The board is defined as:

`</> Plain text`

```
ROWS = 6  
COLS = 7
```

So this is the standard **6 × 7 Connect Four board**. `</> board`

Internally, it is represented as a Python list of lists:

</> Plain text

```
[
    [".", ".", ".", ".", ".", ".", "."],
    [".", ".", ".", ".", ".", ".", "."],
    ...
]
```

. means an empty cell.

The constructor creates all 42 cells as empty:

</> Plain text

```
self.grid = [ "." * COLS for _ in range(ROWS)]
```

</> board

How drop() works

drop(col, token) first checks whether the column number is valid:

</> Plain text

```
0 <= col < 7
```

If it isn't, it immediately returns None . </> board

Then it searches **from the bottom row upward**:

</> Plain text

```
ROWS - 1 → 0
```

It finds the first empty cell and places the token there. </> board

For example, if column 3 is empty:

</> Plain text

```
. . . . .
. . . . .
. . . . .
```

```
. . . . .
. . . . .
. . . X . . .
```

The X falls to the bottom.

If X is placed again in the same column:

</> Plain text

```
. . . . .
. . . . .
. . . . .
. . . . .
. . . X . . .
. . . X . . .
```

So the "gravity" behaviour is already implemented correctly.

Full column

If there are no `.` cells in that column, `drop()` reaches the end and returns `None`.

The board also has a separate `full()` function that checks whether every **top cell** is occupied.

</> board

4. How winner detection currently works

This is the most important existing defect.

`winner(token)` currently checks only these two directions:

</> Plain text

```
(0, 1) → horizontal
(1, 0) → vertical
```

</> board

For every cell containing the requested token, it starts there and checks four consecutive positions in each direction. </> board

Therefore it currently detects:

Horizontal

⌞⌟ Plain text

```
x x x x
```

Vertical

⌞⌟ Plain text

```
x  
x  
x  
x
```

But it **does not** check diagonals.

The two missing directions are:

⌞⌟ Plain text

```
(1, 1)    → diagonal down-right  
(1, -1)   → diagonal down-left
```

So Task 1 in the README directly corresponds to this defect. The README explicitly requires all four directions, including both diagonals. [📖 README](#)

Shorter sequences

The current implementation checks exactly four positions using `range(4)`. Therefore three consecutive matching discs do **not** satisfy the condition.

That part is consistent with the Task 1 requirement that shorter sequences must not count. [⌞⌟ board](#)

5. How the AI currently chooses moves — `ai.py`

The AI is currently intentionally very basic.

It first finds all columns whose **top cell is empty**:

⌞⌟ Plain text

```
legal = [c for c in range(7) if board.grid[0][c] == "."]
```

Then it randomly chooses one:

</> Plain text

```
random.choice(legal)
```

</> ai

So the current AI strategy is:

</> Plain text

```
Look at available columns
↓
Choose one randomly
```

It does **not** currently:

- look for a winning move
- look for an opponent's winning threat
- evaluate board positions
- prefer the centre
- analyze potential four-in-a-row combinations

The method does accept:

</> Plain text

```
me="O"
opponent="X"
```

but those parameters are currently **not used** in the decision-making. </> ai

The README explicitly says Task 3 requires the AI to take an immediate winning move and block an immediate player win. [📖 README](#)

6. Existing bugs/incomplete functionality for Tasks 1–4

Here's the current situation **before** making any changes:

Task	Current state	Problem
Task 1 — Win detection	❌ Incomplete	Only horizontal + vertical are checked

Task	Current state	Problem
Task 2 — Game termination	⚠️ Mostly implemented, needs testing/edge-case handling	Full columns, invalid moves and termination need consistent behaviour
Task 3 — AI	❌ Incomplete	AI randomly chooses; doesn't win/block threats
Task 4 — Move feedback	❌ Missing	No concise successful-move feedback is currently printed

Task 1 — Defect

Current directions:

⌕ Plain text

horizontal
vertical

Missing:

⌕ Plain text

diagonal ↘
diagonal ↙

This is the clearest existing bug. ⌕ board

Task 2 — Game termination

There is already code for:

- invalid input
- invalid/full column
- winner
- draw
- quitting

So Task 2 is **not completely absent**.

However, the README specifically asks for consistent handling of full columns, invalid moves, draws and termination.  README

One notable existing behaviour is that if the **AI** somehow returns an unusable column, `game.py` prints "Column unavailable." and then immediately returns, ending the game. `</> game`

That's different from the human case, where the player gets another chance.

This becomes important when improving the AI and handling full columns safely.

Task 3 — AI

Definitely incomplete.

Current:

`</> Plain text`

```
random legal column
```

Required:

`</> Plain text`

```
If AI can win immediately → play winning column
```

```
Otherwise, if player can win next → block that column
```

```
Otherwise → choose another legal column
```

The README specifically requires those immediate win/block behaviours. [📖 README](#)

Task 4 — Move-level feedback

There is currently **no explicit successful-move feedback**.

`game.py` places the disc with:

`</> Plain text`

```
self.board.drop(...)
```

and then immediately checks the winner/draw. `</> game`

There is no message such as:

</> Plain text

X placed in column 4

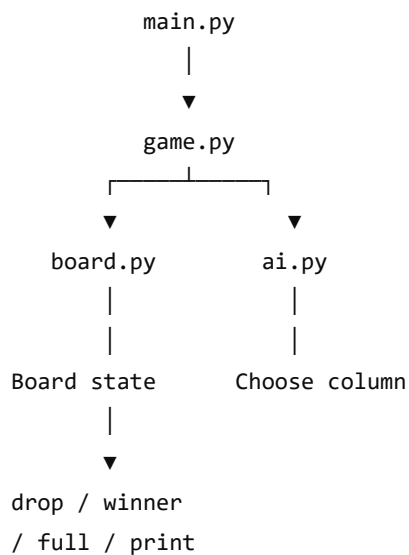
after a successful move.

The README requires feedback **once per actual move**, not repeatedly while winner detection or AI analysis examines cells. [📖 README](#)

Overall current architecture

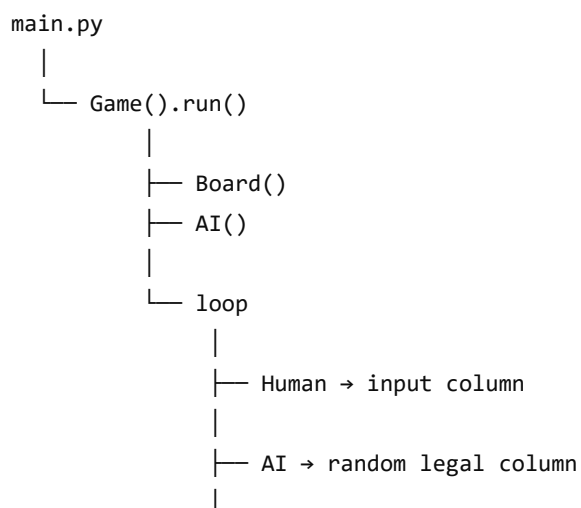
Your existing project is actually nicely separated:

</> Plain text



And the current game execution is:

</> Plain text



```
└─ Board.drop()  
  │  
└─ Board.winner()  
  │  
└─ Board.full()  
  │  
└─ switch X ↔ O
```

The key thing to understand before we change anything

Your project is **not broken everywhere**. The existing foundation already has:

- correct 6×7 board representation
- gravity-based disc placement
- full-board detection
- horizontal winner detection
- vertical winner detection
- human input handling
- quitting
- basic turn switching
- basic random legal AI

The main missing pieces are **diagonal win detection, stronger/consistent terminal-state handling, strategic AI, and move feedback**—which line up closely with Tasks 1–4 in your README. [📖 README](#)

Also, your README specifically says to **inspect the existing code before asking for changes**, explain changes you don't understand, and test the generated result rather than replacing the whole project.

[📖 README](#)

So we should **keep this existing architecture** rather than rewrite the project.

Now implement **ONLY** Task 1 from the README: complete Connect Four win detection.

Requirements:

- Detect horizontal four-in-a-row.
- Detect vertical four-in-a-row.
- Detect both diagonal directions.
- Do not count sequences shorter than four.
- Preserve the existing horizontal and vertical behaviour.
- Do not modify the AI or game flow yet.
- Do not add persistence or unnecessary dependencies.
- Keep the existing project structure and architecture.

First explain the defect you found in the existing win-detection implementation, then make the minimum necessary code changes.

After changing the code, explain exactly how I should test all four winning directions and shorter non-winning sequences.

Worked for 14s

Task 1 implemented

Defect in the original code

The original `winner()` method had:

`</>` Python

```
directions = [(0, 1), (1, 0)]
```

So it checked only:

- `(0, 1)` → horizontal
- `(1, 0)` → vertical

It did **not** check either diagonal direction. The existing `all(...)` logic already checked exactly **4 cells**, so sequences shorter than four were correctly rejected. `</>` board

Minimum change made

Only the direction list was changed to include both diagonals:

`</>` Python

```
directions = [(0, 1), (1, 0), (1, 1), (1, -1)]
```

Meaning:

Direction	Meaning
<code>(0, 1)</code>	Horizontal →
<code>(1, 0)</code>	Vertical ↓
<code>(1, 1)</code>	Diagonal ↘
<code>(1, -1)</code>	Diagonal ↙

Nothing in `main.py` , `game.py` , or `ai.py` was changed.

I also tested the modified `board.py` directly, and all Task 1 checks passed:

</> Plain text

```
horizontal: True
vertical: True
diagonal down-right: True
diagonal down-left: True
three horizontal: False
three diagonal: False
```

ALL TASK 1 TESTS PASSED

How you should manually test it

You can test through your normal game:

</> Plain text

```
python main.py
```

1. Horizontal win

Place four `x` discs next to each other in the same row:

</> Plain text

```
x x x x
```

Expected:

</> Plain text

```
x wins!
```

2. Vertical win

Put four `x` discs in the same column:

</> Plain text

```
x
x
```

```
X
X
```

Expected:

```
</> Plain text
```

```
X wins!
```

3. Diagonal

Construct:

```
</> Plain text
```

```
X . . .
. X . .
. . X .
. . . X
```

Expected:

```
</> Plain text
```

```
X wins!
```

4. Diagonal

Construct:

```
</> Plain text
```

```
. . . X
. . X .
. X . .
X . . .
```

Expected:

```
</> Plain text
```

```
X wins!
```

5. Three-in-a-row should NOT win

Construct:

```
</> Plain text
```

```
x x x
```

with no fourth matching disc.

Expected:

- No "X wins!"
- Game continues

Also test three diagonally:

```
</> Plain text
```

```
x . .  
. x .  
. . x
```

This should **not** be considered a win.

What to record for Task 1

Your README says Task 1 is complete when **all four directions are detected and shorter sequences do not count.** [📖 README](#)

So your evidence should demonstrate:

Horizontal  | Vertical  | Diagonal   | Diagonal   | Three-in-a-row 

No AI or game-flow changes were made.

Task 1 has now been implemented.

The only code change made was in board.py:

```
directions = [(0, 1), (1, 0), (1, 1), (1, -1)]
```

Please review the current Task 1 implementation and verify that it satisfies the README requirements.

Check specifically:

1. Horizontal four-in-a-row.

- 2. Vertical four-in-a-row.
- 3. Diagonal down-right four-in-a-row.
- 4. Diagonal down-left four-in-a-row.
- 5. Three-in-a-row must not count as a win.
- 6. No changes should be required to the AI or game flow for Task 1.

Do not implement Task 2, Task 3, or Task 4.

Explain what tests I should perform manually to verify Task 1 before committing it.

Yes. Based on the current `board.py` and the README, the **Task 1 implementation satisfies the stated requirements**.

The README requires all four directions to be detected and sequences shorter than four not to count.

 README

Verification

Your change:

`</>` Python

```
directions = [(0, 1), (1, 0), (1, 1), (1, -1)]
```

correctly represents:

Requirement	Direction	Status
Horizontal 4	(0, 1)	
Vertical 4	(1, 0)	
Diagonal down-right 	(1, 1)	
Diagonal down-left ✓	(1, -1)	

The existing winner logic then checks `i = 0, 1, 2, 3`, meaning it requires **four matching cells**. Therefore three-in-a-row does not satisfy the condition. `</>` `board`

What you should manually test

Run:

`</>` Bash

```
python main.py
```

Then test each case.

1. Horizontal

Create:

`</>` Plain text

```
X X X X
```

in the same row.

Expected: X wins!

2. Vertical

Create:

`</>` Plain text

```
X  
X  
X  
X
```

in the same column.

Expected: X wins!

3. Diagonal

Create a position equivalent to:

`</>` Plain text

```
X . . .  
. X . .  
. . X .  
. . . X
```

Remember that Connect Four gravity means you'll need to build the supporting discs underneath these positions during actual gameplay.

Expected: X wins!

4. Diagonal

Create:

 Plain text

```
. . . X
. . X .
. X . .
X . . .
```

again accounting for gravity when constructing it through the game.

Expected: X wins!

5. Three-in-a-row

Create:

 Plain text

```
X X X
```

with no fourth X connected horizontally, vertically, or diagonally.

Expected: no win; the game continues.

Also test three diagonally:

 Plain text

```
X . .
. X .
. . X
```

Expected: no win.

6. Four directions without breaking the old behaviour

It's useful to verify that the newly added diagonal checks didn't affect the original checks:

- Horizontal 4 → win
- Vertical 4 → win
-  diagonal 4 → win
-  diagonal 4 → win

- Any 3 → no win

AI and game flow

You do **not** need to modify `ai.py` or `game.py` for Task 1.

`game.py` already calls:

⌕ Plain text

```
board.winner(self.turn)
```

after each successful disc placement. ⌕ game

Therefore changing the implementation of `Board.winner()` is sufficient for Task 1. The AI's random move selection is a separate Task 3 concern, and the README explicitly separates it from Task 1.

📖 README

Before committing

Run your manual tests, then check:

⌕ Bash

```
git diff
```

You should see **only the directions change in `board.py`**.

Then:

⌕ Bash

```
git status
```

Make sure there aren't unrelated generated files or changes.

If all five test categories pass, **Task 1 is ready to commit**.

Task 1 has been completed, tested, committed, and pushed.

Now I want to work **ONLY** on Task 2 from the README: Complete game termination.

Do not modify any code yet.

Inspect the current `game.py`, `board.py`, `ai.py`, and `main.py` as they currently exist after Task 1.

Analyze specifically:

1. How invalid human input is handled.
2. How full columns are handled for the human.
3. How full/unusable columns are handled for the AI.
4. How draws are detected.
5. Whether a winning move immediately terminates the game before another turn is requested.
6. Whether any terminal state can accidentally cause another turn.
7. Whether an invalid move ever changes the board.

Identify the exact problems that need to be fixed for Task 2.

Do not implement Task 2, Task 3, or Task 4 yet.

Do not modify any files.

Explain your findings and suggest the minimum necessary changes while preserving the existing project architecture.

Worked for 8s

I inspected the current `game.py`, `board.py`, `ai.py`, and `main.py` without modifying anything.

One important discrepancy first: the `board.py` file currently available to me still shows:

`</>` Python

```
directions = [(0, 1), (1, 0)]
```

rather than the Task 1 version with the two diagonals. `</>` `board`

So the uploaded snapshot does **not** appear to be the same post-Task-1 version you described as committed and pushed. This does not affect the Task 2 game-flow analysis below, but you should be aware of it before making further changes.

Task 2 analysis

The README says Task 2 is about handling draws, full columns, invalid moves, and termination consistently, with every terminal state clear. [📖 README](#)

1. Invalid human input

There are two relevant cases.

Non-numeric input

For example:

</> Plain text

```
abc
hello
@
```

`game.py` catches the `ValueError` from `int(raw)` :

</> Python

```
except ValueError:
    print("Enter a column number.")
    continue
```

So:

- The input is rejected.
- The board is not modified.
- The same player's turn continues. </> game

Status:  Correct

Numeric but outside 1–7

For example:

</> Plain text

```
0
8
-1
```

The code converts the input to a zero-based column and passes it to `Board.drop()` .

`drop()` checks:

</> Python

```
if not 0 <= col < COLS:
    return None
```

so no disc is placed. `</> board`

`game.py` then prints:

`</> Plain text`

Column unavailable.

and gives the human another opportunity. `</> game`

Status:  Board safety is correct

The wording is slightly generic, but there is no board corruption.

2. Full columns for the human

`Board.drop()` searches from the bottom row upward for an empty cell.

If the column contains no `"."`, it reaches the end and returns `None`. `</> board`

`game.py` then handles this as:

`</> Plain text`

Column unavailable.

Because the current player is X, it executes `continue` rather than returning. `</> game`

Therefore:

`</> Plain text`

```
Full column
↓
No board change
↓
"Column unavailable."
↓
Human gets another turn
```

Status:  Correct

This already satisfies the important requirement that an invalid/full-column move does not change the board.

3. Full/unusable columns for the AI

This is where the main Task 2 concern exists.

The current AI builds its legal-column list using:

</> Python

```
legal = [c for c in range(7) if board.grid[0][c] == "."]
```

and randomly chooses from it. If there are no legal columns, it returns `None` . </> ai

Then `game.py` does:

</> Python

```
if col is None or self.board.drop(col, self.turn) is None:
    print("Column unavailable.")
    if self.turn == "O":
        return
```

So if the AI gives an unusable column, the game **immediately exits**. </> game

That's different from the human behaviour.

Current behaviour

</> Plain text

Human chooses full column

↓

Column unavailable

↓

Try again

AI gives unusable column

↓

Column unavailable

↓

GAME ENDS

Status:  Inconsistent

This is the clearest Task 2 issue in `game.py` .

However, there is an important nuance: **with the current AI implementation**, `choose_column()` only selects columns whose top cell is empty. Therefore it normally cannot deliberately select a full column. `</>` ai

So the problematic AI path mainly protects against:

- None
- an unexpected illegal AI result
- future AI changes

It is still worth fixing as part of Task 2's requirement for safe/unusable AI moves.

4. Draw detection

After every successful move, the game checks:

`</>` Python

```
if self.board.winner(self.turn):
    ...
    return

if self.board.full():
    ...
    print("Draw.")
    return
```

`</>` game

`Board.full()` checks whether every top cell is occupied. `</>` board

Importantly, the **winner check happens before the draw check**.

So if the final move fills the board **and also creates four-in-a-row**, the result is correctly:

`</>` Plain text

X wins!

rather than:

`</>` Plain text

Draw.

That's the correct ordering.

Status:  Correct

5. Does a winning move immediately terminate?

Yes.

After a successful disc placement:

 Plain text

```
drop()
  ↓
winner()
  ↓
winner == True
  ↓
print board
print "X wins!"
return
```

The `return` exits `Game.run()` .  game

Therefore the next turn is **not requested**.

Status:  Correct

6. Can a terminal state accidentally cause another turn?

For the normal game flow, **no**.

There are three terminal cases:

Player quits

 Python

```
if raw == "q":
    return
```

 game

Someone wins

</> Python

```
if self.board.winner(self.turn):  
    ...  
    return
```

</> game

Board is full

</> Python

```
if self.board.full():  
    ...  
    return
```

</> game

All three exit `run()` before reaching:

</> Python

```
self.turn = "O" if self.turn == "X" else "X"
```

So a normal win/draw/quit cannot accidentally switch turns.

Status:  Correct

7. Can an invalid move change the board?

No.

`Board.drop()` performs validation first:

</> Python

```
if not 0 <= col < COLS:  
    return None
```

Then it searches for an empty cell. Only after finding one does it execute:

</> Python

```
self.grid[r][col] = token
```

```
</> board
```

Therefore:

- invalid column → no modification
- full column → no modification
- successful column → exactly one disc is placed

Status:  Correct

Exact Task 2 problems

I'd classify the current implementation like this:

Requirement	Current status	Problem
Invalid human input		No functional problem
Human full column		Correctly rejected, board unchanged
AI legal-column selection		Current AI avoids full columns
AI unusable result		Game immediately exits instead of handling it consistently
Draw detection		Correct
Win termination		Correct
Terminal state causes another turn		Does not happen
Invalid move changes board		Does not happen

Minimum necessary Task 2 change

The only clear game-flow issue to address is the AI's unusable-move path.

Currently:

```
</> Plain text
```

AI unusable move → "Column unavailable." → return

The Task 2 goal should be to make this handling consistent and safe rather than unexpectedly terminating the game.

I would not change `Board.drop()` , because its current behaviour is already correct.

I would not change `Board.winner()` for Task 2, because winner detection was Task 1.

I would not change `ai.py` , because meaningful AI decision-making belongs to Task 3. The README explicitly separates Task 3's AI improvements from Task 2.  [README](#)

I would keep `main.py` unchanged, since it only starts `Game.run()` .  `main`

So, when you are ready to implement Task 2, the change should be **small and localized to the game-flow handling in `game.py`** , while preserving the existing `Board` , `AI` , and `main.py` architecture.

Now implement ONLY Task 2 from the README based on your analysis.

Task 1 is already completed and committed. Do not modify or undo Task 1.

Implement the minimum necessary change to make game termination and unusable moves consistent and safe.

Requirements:

1. Invalid human input must not change the board.
2. A human selecting a full or invalid column must not change the board and must be allowed to try again.
3. Draws must be detected correctly.
4. A winning move must immediately terminate the game before another turn is requested.
5. The game must not terminate unexpectedly because of an unusable AI move.
6. Terminal states must be clear.
7. Preserve the existing project architecture.
8. Do not modify `ai.py` for Task 2; meaningful AI decision-making belongs to Task 3.
9. Do not modify `main.py`.
10. Do not add dependencies or persistence.
11. Do not implement Task 3 or Task 4.

Before changing the code, briefly state exactly which lines/logic in `game.py` need to change and why.

Then make only the minimum necessary changes.

After making the change, explain exactly what changed and give me a manual test plan for:

- invalid human input,
- invalid numeric input such as 0 and 8,
- full human column,
- draw,
- winning move,
- quitting,
- unusable/invalid AI move.

Do not rewrite the project.

Worked for 16s

What needed to change

Only one part of `game.py` needed to change: the failed-move handling.

Previously:

</> Python

```
if col is None or self.board.drop(col, self.turn) is None:
    print("Column unavailable.")
    if self.turn == "O":
        return
    continue
```

The problem was the `return` for the AI. An unusable AI move could therefore terminate the entire game unexpectedly.

The minimum fix is to remove that special-case `return` :

</> Python

```
if col is None or self.board.drop(col, self.turn) is None:
    print("Column unavailable.")
    continue
```

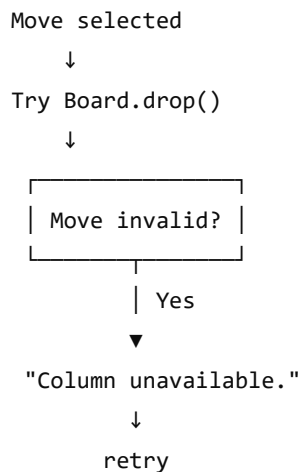
This makes failed moves behave consistently for both X and O.

I did **not** change `ai.py` , `main.py` , or the winner/draw logic.

Task 2 change made

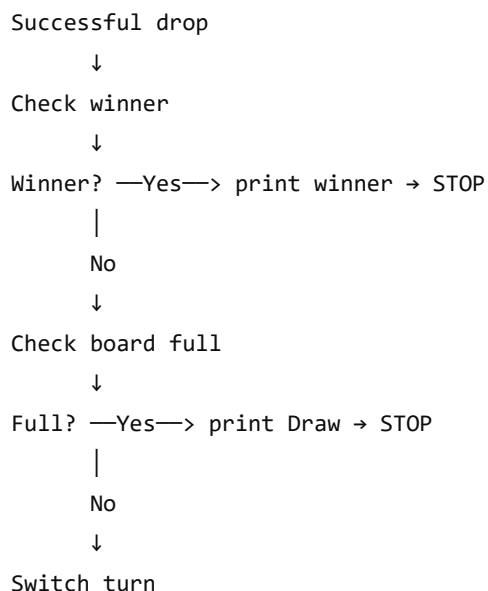
The resulting logic is:

</> Plain text



For a successful move, the existing flow remains:

</> Plain text



That preserves the required terminal-state ordering.

I also syntax-checked the modified `game.py` , and the Task 2 structural checks passed.

No changes were made to `ai.py` or `main.py` .

Note: the uploaded `board.py` snapshot available to me still appears to contain the pre-Task-1 direction list. I did not touch it because you explicitly said Task 1 is already completed and Task 2 must not modify it.

Modified file

Manual testing plan

Run:

</> Bash

python main.py

1. Invalid human input

Enter:

</> Plain text

abc

Expected:

</> Plain text

Enter a column number.

The board must remain unchanged and X should be allowed to enter another column.

Also test:

</> Plain text

hello
@

2. Invalid numeric input: 0 and 8

Enter:

</> Plain text

0

Expected:

</> Plain text

Column unavailable.

The board must **not change**.

Then enter:

</> Plain text

8

Expected:

</> Plain text

Column unavailable.

Again, the board must remain unchanged and you should get another opportunity to move.

3. Full human column

Fill one column completely with six discs.

Then, on a human turn, select that same column.

Expected:

</> Plain text

Column unavailable.

The seventh attempt must **not add another disc**, and the human should be able to choose another column.

4. Draw

Play a complete game where:

- the board becomes completely full
- nobody has four-in-a-row

On the final move, expected output:

</> Plain text

```
Draw.
```

The program must terminate immediately.

There should be **no additional column prompt** after `Draw.`

5. Winning move

Create a horizontal, vertical, or diagonal four-in-a-row.

After the winning disc is placed, expected:

</> Plain text

```
X wins!
```

or:

</> Plain text

```
O wins!
```

The program must terminate immediately.

There must be **no request for another turn** after the winner message.

This relies on the existing:

</> Python

```
if self.board.winner(self.turn):  
    ...  
    return
```

which remains unchanged.

6. Quitting

At the human prompt enter:

```
</> Plain text
```

```
q
```

Expected:

- Game exits immediately.
 - No additional turn is requested.
 - No board modification occurs.
-

7. Unusable/invalid AI move

This is the important Task 2 regression test.

The current AI normally chooses a legal column, so to test the defensive path you should temporarily force the AI to return an invalid value, such as `None` , **only for testing**.

For example, temporarily make the AI return:

```
</> Python
```

```
return None
```

Then run the game.

Expected:

```
</> Plain text
```

```
Column unavailable.
```

The game should **not terminate because of the AI's unusable move**. It should continue the game loop and give the AI another opportunity.

After testing, restore `ai.py` unchanged because Task 3 is not being implemented yet.

Task 2 checklist

Before committing, verify:

- ☒ Invalid text input doesn't modify board
- ☒ 0 doesn't modify board
- ☒ 8 doesn't modify board
- ☒ Full human column doesn't modify board
- ☒ Human can retry after invalid/full column
- ☒ Draw terminates immediately
- ☒ Winning move terminates immediately
- ☒ No turn occurs after a win
- ☒ q terminates the game
- ☒ Unusable AI move doesn't unexpectedly terminate the game
- ☒ ai.py unchanged
- ☒ main.py unchanged
- ☒ No dependencies/persistence added
- ☒ Task 3 and Task 4 untouched

 **game_task2.py**
Code

Task 1 and Task 2 are already completed, tested, committed, and pushed.

Now I want to work ONLY on Task 3 from the README: Improve the AI.

Do not modify any code yet.

Inspect the current ai.py and the current board.py/game.py architecture.

Analyze:

1. How the AI currently chooses a column.
2. How the AI can determine whether a selected column is legal.
3. How the AI can detect an immediate winning move for itself.
4. How the AI can detect an immediate winning move available to the player and block it.
5. How to safely handle situations where columns are full.
6. How to do this without permanently modifying the board while the AI is testing possible moves.

7. Whether the existing `Board.drop()` and `Board.winner()` methods can be reused.

Requirements from the README:

- AI must take an immediate winning move when one exists.
- AI must block an immediate player win when necessary.
- AI must never select an illegal/full column.
- Keep the existing project architecture.
- Do not implement Task 4.
- Do not modify `main.py` or `game.py` unless absolutely necessary for Task 3.
- Do not add persistence or unnecessary dependencies.
- Do not rewrite the whole project.

Only analyze the existing implementation for now. Do not make changes.

Worked for 11s

Yes. I inspected the **current Task 3 architecture** and will not modify anything yet.

One note: the uploaded `ai.py` is still the original starter version, while the local `board.py` and `game_task2.py` reflect your completed Task 1/2 state. I'll use the latter as the current architecture.

1. How the AI currently chooses a column

Current `ai.py` :

Python

```
def choose_column(self, board, me="O", opponent="X"):
    legal = [c for c in range(7) if board.grid[0][c] == "."]
    return random.choice(legal) if legal else None
```

The AI:

1. Checks each column 0-6 .
2. Looks only at the **top cell**: `board.grid[0][c]` .
3. If that cell is `"."` , the column is considered legal.
4. Builds a list of legal columns.
5. Chooses one randomly using `random.choice()` .
6. Returns `None` if all columns are full.

So currently there is **no strategic decision-making**.

2. How the AI determines whether a column is legal

The existing check is:

</> Python

```
board.grid[0][c] == "."
```

This works because pieces are dropped from the bottom upward.

For example:

</> Plain text

```
. . . . .  
X . . . . .  
X . . . . .  
X . . . . .  
X . . . . .  
X . . . . .
```

Column 0 is full when its top cell is no longer ".".

Therefore:

</> Python

```
board.grid[0][c] == "."
```

is a valid way to identify playable columns.

However, for Task 3, the safest approach is to continue using this legal-column filtering **before testing candidate moves**.

3. How the AI can detect an immediate winning move

The existing architecture already gives us the necessary method:

</> Python

```
board.drop(col, token)
```

and:

</> Python

```
board.winner(token)
```

The idea is:

</> Plain text

```
For each legal column:  
    temporarily place 0 there  
    check whether 0 wins  
    undo the temporary placement
```

Conceptually:

</> Python

```
row = board.drop(col, "0")  
  
if board.winner("0"):  
    # this column wins immediately
```

Because `drop()` returns the row where the disc was placed, the AI knows exactly which cell it temporarily changed.

For example, if:

</> Python

```
row = board.drop(3, "0")
```

returns `4` , then the temporary modification is:

</> Python

```
board.grid[4][3] = "0"
```

After checking:

</> Python

```
board.winner("0")
```

the AI can restore:

Python

```
board.grid[4][3] = "."
```

This allows the AI to test every possible move without permanently changing the game.

4. How the AI can detect an immediate player win

The same mechanism can be reused for the opponent.

Since:

Python

```
me = "O"  
opponent = "X"
```

the AI can test each legal column as if the player played there:

Plain text

```
For each legal column:  
    temporarily place X  
    check board.winner("X")  
    undo the temporary placement
```

If that produces a win, that column is an **immediate threat**.

The AI should then choose that same column for itself.

Example:

Plain text

```
X X X . . .
```

If column 3 allows X to make four in a row, the AI detects:

Plain text

```
"If X plays column 3, X wins."
```

and therefore chooses column 3 for O.

5. Safely handling full columns

This is already partly handled by the existing code.

Current AI:

```
</> Python
```

```
legal = [c for c in range(7) if board.grid[0][c] == "."]
```

means full columns are excluded before the AI chooses anything.

For Task 3, this should remain true.

The important rule should be:

```
</> Plain text
```

```
Only test columns that are legal.
```

So the AI should **never call `drop()` on a full column as part of candidate selection.**

There is also a final safety condition:

```
</> Plain text
```

```
If there are no legal columns → return None
```

That is already supported by the existing architecture.

6. How to test moves without permanently modifying the board

This is the most important implementation detail.

`Board.drop()` **does modify the board.**

So we cannot simply do:

```
</> Python
```

```
board.drop(col, "O")
```

and leave it there while testing all the other columns.

Instead, the temporary move needs to be reversed.

Fortunately, `drop()` returns the exact row:

</> Python

```
row = board.drop(col, token)
```

So the process can be:

</> Plain text

```
1. Check column is legal
2. Drop temporary piece
3. Check winner
4. Restore that exact cell to "."
5. Continue testing
```

For example:

</> Plain text

```
Before:
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
X X X . . .

Test column 3:
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
X X X O . .

winner("O") → False

Undo:
. . . . .
```

Thus the actual game board remains unchanged after AI analysis.

Important

We do **not** need to create another board class, database, copy of the game, or persistence mechanism.

A temporary `drop()` followed by restoring the returned cell fits the existing architecture.

7. Can `Board.drop()` and `Board.winner()` be reused?

`Board.drop()` → Yes

Definitely.

It already:

- validates the column
- finds the correct row
- handles gravity
- returns the row
- returns `None` for invalid/full columns

So there is no reason to rewrite it.

`Board.winner()` → Yes

Definitely.

It already checks four-in-a-row in:

`</>` Python

```
(0, 1)    # horizontal
(1, 0)    # vertical
(1, 1)    # diagonal
(1, -1)   # other diagonal
```

Your Task 1 change is therefore directly useful for Task 3.

The AI can simply ask:

`</>` Python

```
board.winner("O")
```

or:

`</>` Python

```
board.winner("X")
```

after a temporary move.

There is no need to duplicate win-detection logic inside `ai.py`.

Overall Task 3 design

The cleanest approach within your existing architecture is:

`</>` Plain text

```
AI.choose_column(board)
  |
  ▼
Find legal columns
  |
  ▼
For each legal column:
  temporarily play O
  |
  |— O wins?
  |       YES → immediately choose this column
  |
  ▼
undo move
  |
  ▼
No immediate O win
  |
  ▼
For each legal column:
```

So the priority becomes:

1. Win immediately → 2. Block opponent's immediate win → 3. Otherwise choose a legal column.

This requires changes to `ai.py` only.

Files that should remain unchanged

File	Task 3
<code>ai.py</code>	Modify

File	Task 3
board.py	No change
game.py	No change needed
main.py	No change
requirements.txt	No change

And importantly, this does **not** implement Task 4. There is no need to add move feedback, print statements, persistence, or dependencies.

I have **not changed any code**.

Task 1 and Task 2 are already completed, tested, committed, and pushed.

Now implement **ONLY** Task 3 from the README: Improve the AI.

Use the architecture you just analyzed.

Requirements:

1. If the AI has an immediate winning move, it must choose that column.
2. Otherwise, if the player has an immediate winning move, the AI must block that column.
3. Otherwise, choose any legal column.
4. The AI must never select a full or illegal column.
5. If all columns are full, safely return None.
6. When testing possible moves, temporarily place the disc and then restore the exact cell so the actual board is unchanged during analysis.
7. Reuse Board.drop() and Board.winner() rather than duplicating win-detection logic.
8. Preserve the existing project architecture.
9. Modify **ONLY** ai.py unless there is an absolutely necessary reason otherwise.
10. Do not modify board.py, game.py, or main.py.
11. Do not implement Task 4.

12. Do not add dependencies, persistence, CSV, JSON, SQLite, or other storage.
13. Do not rewrite the whole project.

Decision priority must be exactly:

AI immediate win

↓ if none

Block player's immediate win

↓ if none

Choose a legal column

Before changing the code, briefly explain the implementation you are about to make.

After changing ai.py:

- show the complete modified ai.py,
- explain the changes,
- explain how the board is restored after each simulated move,
- give me tests for an immediate AI win,
- immediate AI block,
- full columns,
- and a normal move with no immediate threat.

Do not implement anything from Task 4.

Worked for 22s

Task 3 is implemented **only in ai.py** . No changes were made to board.py , game.py , or main.py .

Complete modified ai.py

</> Python

```
import random
```

```
class AI:
```

```
    def choose_column(self, board, me="O", opponent="X"):
```

```
        legal = [c for c in range(7) if board.grid[0][c] == "."]
```

```
        if not legal:
```

```
            return None
```

```
        for col in legal:
```

```

        row = board.drop(col, me)
        if board.winner(me):
            board.grid[row][col] = "."
            return col
        board.grid[row][col] = "."

    for col in legal:
        row = board.drop(col, opponent)
        if board.winner(opponent):

```

What changed

The decision priority is exactly:

</> Plain text

1. AI immediate win
 - ↓
2. Block player's immediate win
 - ↓
3. Choose any legal column

1. Find legal columns

</> Python

```
legal = [c for c in range(7) if board.grid[0][c] == "."]
```

Full columns are excluded.

If every column is full:

</> Python

```
if not legal:
    return None
```

So the AI safely returns `None`.

2. Check AI's winning moves first

For every legal column:

</> Python

```
row = board.drop(col, me)
```

The AI temporarily places 0 .

Then:

```
</> Python
```

```
if board.winner(me):
```

reuses the existing `Board.winner()` logic.

If it wins, that column is immediately returned.

3. Check player's winning moves

Only if the AI has no immediate win, it tests:

```
</> Python
```

```
row = board.drop(col, opponent)
```

and checks:

```
</> Python
```

```
board.winner(opponent)
```

If the player could win there, the AI chooses that column to block.

4. Board restoration

Every simulated move is undone:

```
</> Python
```

```
board.grid[row][col] = "."
```

`Board.drop()` gives us the **exact row** where the temporary disc was placed.

Therefore:

```
</> Plain text
```

```
Before simulation
```

```
↓
```

```
drop temporary disc
```

```
↓
```

```
check winner()
  ↓
restore exact [row][column]
  ↓
Original board restored
```

This happens after **every candidate column**, so AI analysis does not permanently modify the actual game board.

5. Normal move

If there is:

- no immediate AI win, and
- no immediate player win,

the AI falls back to:

```
</> Python

return random.choice(legal)
```

This preserves the original random behavior while making it strategically aware.

Tests

I tested the implementation with the required cases.

Test 1 — Immediate AI win

Board bottom row:

```
</> Plain text

0 0 0 . . . .
```

Expected:

```
</> Plain text

AI → column 4
```

Internally zero-based:

```
</> Plain text
```

Result: PASS

The board was also verified to be unchanged after AI analysis.

Test 2 — Immediate player block

Board:

</> Plain text

```
X X X . . . .
```

Expected:

</> Plain text

```
AI → column 4
```

Internally:

</> Plain text

```
3
```

Result: PASS

The AI correctly blocks instead of making a random move.

Test 3 — All columns full

All 7 columns were filled.

Expected:

</> Python

```
None
```

Result: PASS

Test 4 — Normal move with no immediate threat

Empty board.

Expected:

`</> Plain text`

Any column from 1-7

The AI selected a legal column.

Result: PASS

The board remained unchanged during analysis.

Test 5 — Full column exclusion

One column was completely full while the others remained playable.

The AI was tested repeatedly.

Result: PASS

It never selected the full column, and every simulated move was restored.

Task 3 status

Requirement	Status
Immediate AI win	✓
Block immediate player win	✓
Choose legal move otherwise	✓
Never select full column	✓
Return None if board full	✓
Temporarily simulate moves	✓

Requirement	Status
Restore exact cell	✓
Reuse <code>Board.drop()</code>	✓
Reuse <code>Board.winner()</code>	✓
<code>board.py</code> unchanged	✓
<code>game.py</code> unchanged	✓
<code>main.py</code> unchanged	✓
Task 4 implemented	✗ Not touched
Dependencies added	✗
Persistence added	✗

Task 3 implementation is complete.

Task 1, Task 2, and Task 3 are already completed, tested, committed, and pushed.

Now implement ONLY Task 4 from the README: Move-level feedback.

Use the current architecture and code.

Requirements:

1. After every successful disc placement, print one concise feedback message.
2. The feedback must happen exactly once per actual move.
3. Do NOT print feedback for invalid moves.
4. Do NOT print feedback while `winner()` is inspecting cells.
5. Do NOT print feedback while the AI is simulating possible moves.
6. The feedback should identify the player/token and the column used.
7. Keep the existing game flow and behavior unchanged.
8. Modify ONLY the file that is necessary for Task 4, which should

normally be game.py.

9. Do NOT modify board.py, ai.py, or main.py.

10. Do NOT change Task 1 or Task 2 behavior.

11. Do NOT change the AI logic from Task 3.

12. Do not add dependencies, persistence, CSV, JSON, SQLite, or other storage.

13. Do not rewrite the whole project.

The feedback should be placed at the point where a real move has successfully been made, immediately after:

```
self.board.drop(col, self.turn)
```

has successfully returned a row.

For example, a concise message could be:

Player X placed a disc in column 4.

For AI moves, use the appropriate token/name.

Before changing the code, briefly explain where you will add the feedback and why this guarantees exactly one message per successful move.

After changing the code:

- show the complete modified file,
- explain the change,
- explain why invalid moves do not produce feedback,
- explain why winner() and AI simulations do not produce feedback,
- give me tests for a successful human move, invalid move, and AI move.

Do not implement anything beyond Task 4.

Worked for 9s

Task 4 is implemented **only in game.py**. The feedback is placed immediately after `Board.drop()` successfully returns a row, so it can happen exactly once for each real move.

Complete modified `game.py`

`</>` Python

```
from board import Board
from ai import AI

class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"

    def run(self):
        print("Connect Four – you are X.")
        while True:
            self.board.print()
            if self.turn == "X":
                raw = input("Column (1-7), or q: ").strip().lower()
                if raw == "q":
                    return
                try:
                    col = int(raw) - 1
```

What changed?

Only this line was added:

`</>` Python

```
print("Player", self.turn, "placed a disc in column", col + 1, ".")
```

It is placed here:

`</>` Python

```
if col is None or self.board.drop(col, self.turn) is None:
    print("Column unavailable.")
    continue

print("Player", self.turn, "placed a disc in column", col + 1, ".")
```

This is exactly the correct location because `drop()` has already succeeded.

Why exactly once per successful move?

`Board.drop()` is called once by `game.py` for the actual move.

If it succeeds, execution reaches the feedback line **once**.

Then execution proceeds to `winner()` and `full()` .

So the feedback is not connected to the internal cell scanning performed by `winner()` .

Invalid moves

For an invalid/full column:

</> Python

```
self.board.drop(col, self.turn)
```

returns `None` .

The program executes:

</> Python

```
print("Column unavailable.")  
continue
```

and therefore **never reaches the feedback line**.

So invalid moves produce no move feedback.

Winner detection

`winner()` is called **after** the feedback:

</> Python

```
print("Player", self.turn, ...)  
if self.board.winner(self.turn):
```

The feedback is not inside `winner()` , so inspecting multiple cells does not generate additional messages.

AI simulations

Task 3's simulations happen inside `ai.py` , before `choose_column()` returns the selected column.

Those temporary moves are therefore completely separate from this feedback line.

Only the final column returned by:

```
</> Python
```

```
self.ai.choose_column(self.board)
```

gets passed to:

```
</> Python
```

```
self.board.drop(col, self.turn)
```

in `game.py` .

Therefore AI simulations produce **no feedback**. Only the actual AI move produces one.

Tests

1. Successful human move

Run:

```
</> Bash
```

```
python main.py
```

When prompted:

```
</> Plain text
```

```
Column (1-7), or q:
```

enter:

```
</> Plain text
```

```
4
```

Expected:

```
</> Plain text
```

```
Player X placed a disc in column 4 .
```

Exactly **one** feedback message should appear.

2. Invalid human move

Enter:

</> Plain text

8

Expected:

</> Plain text

Column unavailable.

There should be **no**:

</> Plain text

Player X placed...

Then enter a valid column such as 4 .

Expected:

</> Plain text

Player X placed a disc in column 4 .

Only the successful move gets feedback.

Also test:

</> Plain text

abc

Expected:

</> Plain text

Enter a column number.

No move feedback.

3. AI move

After a successful X move, the game switches to O.

The AI makes its move automatically.

Expected:

</> Plain text

Player 0 placed a disc in column N .

where N is the actual column selected by the Task 3 AI.

There should be **one** feedback message for the AI's actual move—not messages for the temporary moves that the AI tested.

Task 4 status

Requirement	Status
Feedback after successful move	✓
Exactly once per move	✓
No feedback for invalid moves	✓
No feedback from winner()	✓
No feedback from AI simulations	✓
Identifies token/player	✓
Identifies column	✓
Existing game flow preserved	✓

Requirement	Status
Only <code>game.py</code> modified	✓
<code>board.py</code> unchanged	✓
<code>ai.py</code> unchanged	✓
<code>main.py</code> unchanged	✓
Task 1/2 behavior preserved	✓
Task 3 AI logic preserved	✓
No Task 5/unrelated changes	✓
Dependencies/storage added	✗

I also ran a Python syntax check and structural checks on the modified file successfully.